

Task 12: Session 12 (AIML) - Assignment Questions

Name: Mihir Sopan Patil

Domain: AIML

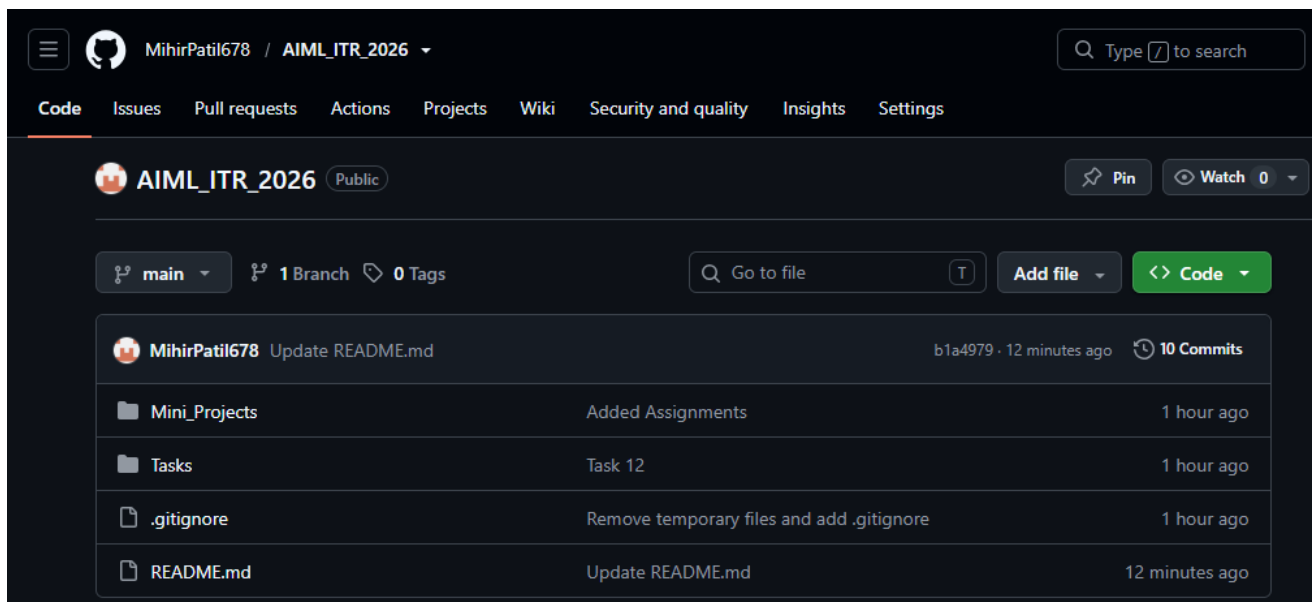
College & Branch Name: Zeal Polytechnic, Diploma In AIML

GitHub: https://github.com/MihirPatil678/AIML_ITR_2026/tree/main/Tasks/Task_12

1. GitHub Account Setup

- Create a GitHub account (if you don't have one).
- Create a new repository named Python-Basic-Projects.
- After implementing each feature in each question's solution, commit your code with a proper commit message (e.g., "Implement Loop for Counting Vowels" or "Create analyze_string function").
- After completing the full project, push everything to GitHub.

Output:



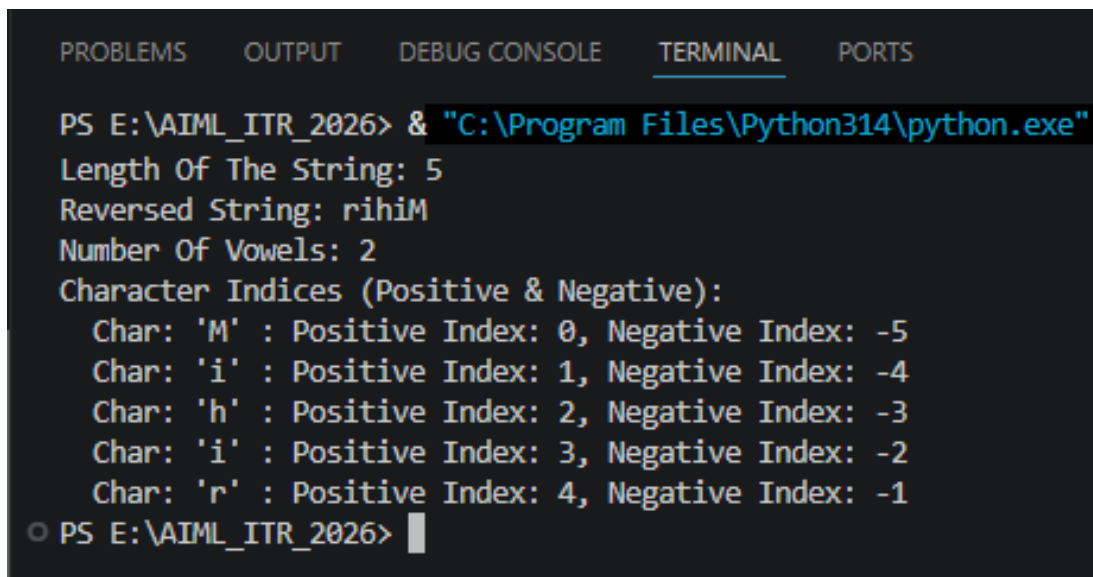
2. Strings + Loops + Functions

Write a function `analyze_string(s)` that takes a string as input and:

- Prints its length using `len()`.
- Prints the string in reverse using slicing.
- Counts and prints how many vowels (a,e,i,o,u) are in the string (case insensitive).
- Uses a for loop with `range()` to print each character with its positive and negative index.

Call this function with user input and handle empty string case.

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\AIML_ITR_2026> & "C:\Program Files\Python314\python.exe"
Length Of The String: 5
Reversed String: rihiM
Number Of Vowels: 2
Character Indices (Positive & Negative):
  Char: 'M' : Positive Index: 0, Negative Index: -5
  Char: 'i' : Positive Index: 1, Negative Index: -4
  Char: 'h' : Positive Index: 2, Negative Index: -3
  Char: 'i' : Positive Index: 3, Negative Index: -2
  Char: 'r' : Positive Index: 4, Negative Index: -1
PS E:\AIML_ITR_2026> |
```

Program Code:

```
def analyze_string(s):
    # Handle Empty String Case First
    if s == "":
        print("Error: The String Is Empty. Please Provide A Non-Empty String.")
        return

    # Length Of The String
    length = len(s)
    print(f"\nLength Of The String: {length}")

    # Reverse The String Using Slicing
    reversed_s = s[::-1]
    print(f"Reversed String: {reversed_s}")

    # Count Vowels (Case Insensitive)
```

```

vowels = "aeiouAEIOU"
vowel_count = 0
for j in s:
    if j in vowels:
        vowel_count += 1
print(f"Number Of Vowels: {vowel_count}")

# Print Each Character With Its Positive & Negative Index
print("Character Indices (Positive & Negative):")
for i in range(length):
    neg_index = i - length # equivalent negative index
    print(f" Char: '{s[i]}' : Positive Index: {i}, Negative Index:
{neg_index}")

def main():
    user_input = input("Enter A String To Analyze: ")
    analyze_string(user_input)

if __name__ == "__main__":
    main()

```

3. Lists + Functions + Exception Handling

Create a function `manage_marks()` that:

- Takes 5 subject marks as input from the user.
- Stores them in a list.
- Handles `ValueError` if non-numeric input is given.
- Calculates and prints the average, highest, and lowest marks.
- Sorts the list in descending order and prints it.

Use meaningful comments.

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS E:\AIML_ITR_2026> & "C:\Program Files\Python314\python.exe" e:/AIML_ITR_2026
Enter Marks For Subject 1: 76
Enter Marks For Subject 2: 29
Enter Marks For Subject 3: 42
Enter Marks For Subject 4: 62
Enter Marks For Subject 5: 58

Marks Entered: [76.0, 29.0, 42.0, 62.0, 58.0]
Average Marks: 53.40
Highest Marks: 76.0
Lowest Marks: 29.0
Marks Sorted In Descending Order: [76.0, 62.0, 58.0, 42.0, 29.0]
○ PS E:\AIML_ITR_2026> 
```

Program Code:

```
def manage_marks():
    marks = [] # List To Store The 5 Subject Marks
    subjects = 5

    # Loop Until We Collect 5 Valid Numeric Marks
    i = 1
    while i <= subjects:
        try:
            value = float(input(f"Enter Marks For Subject {i}: "))
            marks.append(value)
            i += 1 # Only Move To Next Subject If Input Was Valid
        except ValueError:
            # Handle Non-Numeric Input Gracefully & Re-Prompt
```

```
        print("Invalid Input! Please Enter A Numeric Value.")

# Calculate Average, Highest, & Lowest Using Built-In Functions
average = sum(marks) / len(marks)
highest = max(marks)
lowest = min(marks)

print(f"\nMarks Entered: {marks}")
print(f"Average Marks: {average:.2f}")
print(f"Highest Marks: {highest}")
print(f"Lowest Marks: {lowest}")

# Sort The List In Descending Order
marks.sort(reverse=True)
print(f"Marks Sorted In Descending Order: {marks}")

if __name__ == "__main__":
    manage_marks()
```

4. OOP + Lists + Exception Handling

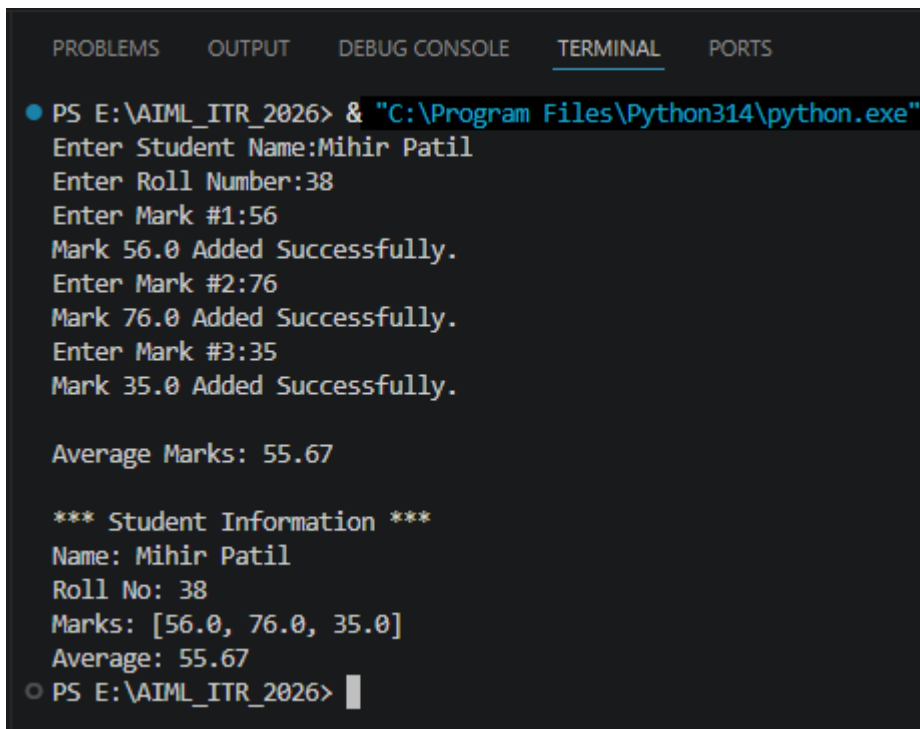
Create a class Student with attributes: name, roll_no, and marks_list (a list of marks).

Include methods:

- `__init__` to initialize the student.
- `add_mark(mark)` to add a mark to the list (handle invalid marks).
- `get_average()` to return the average.
- `display_info()` to show all details.

Create one student object, add marks using user input, and demonstrate all methods with exception handling.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS E:\AIML_ITR_2026> & "C:\Program Files\Python314\python.exe"
Enter Student Name:Mihir Patil
Enter Roll Number:38
Enter Mark #1:56
Mark 56.0 Added Successfully.
Enter Mark #2:76
Mark 76.0 Added Successfully.
Enter Mark #3:35
Mark 35.0 Added Successfully.

Average Marks: 55.67

*** Student Information ***
Name: Mihir Patil
Roll No: 38
Marks: [56.0, 76.0, 35.0]
Average: 55.67
○ PS E:\AIML_ITR_2026> █
```

Program Code:

```
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no
        self.marks_list = [] # Start With An Empty List of marks

    def add_mark(self, mark):
        try:
            mark = float(mark)
```

```

        if mark < 0 or mark > 100:
            # Basic Check For A Mark Range
            print("Invalid Mark! Marks Must Be Between 0 & 100.")
            return
        self.marks_list.append(mark)
        print(f"Mark {mark} Added Successfully.")
    except ValueError:
        print("Invalid Mark! Please Enter A Numeric Value.")

def get_average(self):
    if not self.marks_list:
        return 0
    return sum(self.marks_list) / len(self.marks_list)

def display_info(self):
    print("\n*** Student Information ***")
    print(f"Name: {self.name}")
    print(f"Roll No: {self.roll_no}")
    print(f"Marks: {self.marks_list}")
    print(f"Average: {self.get_average():.2f}")

def main():
    # Create One Student Object
    name = input("Enter Student Name:")
    roll_no = input("Enter Roll Number:")
    student = Student(name, roll_no)

    # Add Marks Using User Input, Demonstrating Exception Handling
    num_marks = 3
    for i in range(num_marks):
        mark_input = input(f"Enter Mark #{i + 1}:")
        student.add_mark(mark_input)

    # Demonstrate All Methods
    print(f"\nAverage Marks: {student.get_average():.2f}")
    student.display_info()

if __name__ == "__main__":
    main()

```

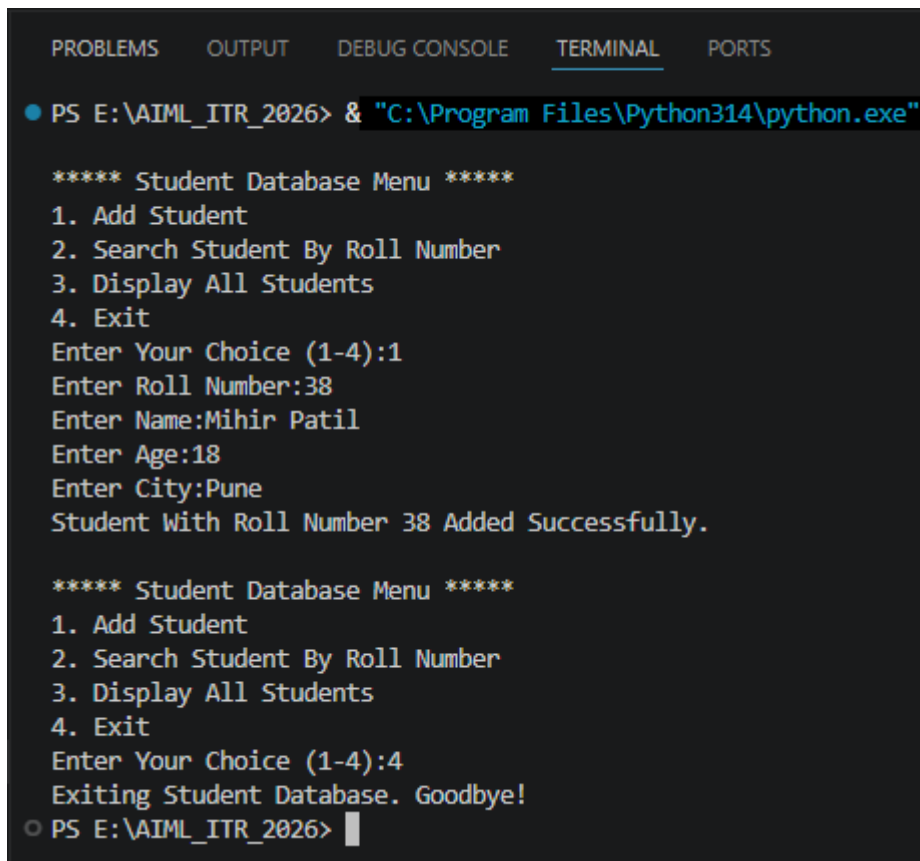
5. Dictionaries + Functions + Control Structures

Write a function `student_database()` that uses a dictionary (roll number as key) to store student records.

- Provide a menu using while loop:
- Add student (name, age, city)
- Search student by roll number
- Display all students
- Exit

Use `get()`, `update()`, and proper exception handling for invalid inputs.

Output:



```
PS E:\AIML_ITR_2026> & "C:\Program Files\Python314\python.exe"
**** Student Database Menu ****
1. Add Student
2. Search Student By Roll Number
3. Display All Students
4. Exit
Enter Your Choice (1-4):1
Enter Roll Number:38
Enter Name:Mihir Patil
Enter Age:18
Enter City:Pune
Student With Roll Number 38 Added Successfully.

**** Student Database Menu ****
1. Add Student
2. Search Student By Roll Number
3. Display All Students
4. Exit
Enter Your Choice (1-4):4
Exiting Student Database. Goodbye!
PS E:\AIML_ITR_2026> 
```

Program Code:

```
def student_database():
    records = {} # dictionary: roll_no -> {name, age, city}

    while True:
        print("\n**** Student Database Menu ****")
```



```

print("1. Add Student")
print("2. Search Student By Roll Number")
print("3. Display All Students")
print("4. Exit")

try:
    choice = int(input("Enter Your Choice (1-4):"))
except ValueError:
    print("Invalid Input! Please Enter A Number Between 1 & 4.")
    continue

if choice == 1:
    try:
        roll_no = int(input("Enter Roll Number:"))
        name = input("Enter Name:")
        age = int(input("Enter Age:"))
        city = input("Enter City:")

        # update() Is Used To Add/Overwrite The Record For This roll_no
        records.update({roll_no: {"name": name, "age": age, "city": city}})
        print(f"Student With Roll Number {roll_no} Added Successfully.")
    except ValueError:
        print("Invalid Age Entered! Student Not Added.")

elif choice == 2:
    roll_no = int(input("Enter Roll Number To Search:"))
    # get() Returns None If The Key Doesn't Exist, Avoiding A KeyError
    student = records.get(roll_no)
    if student:
        print(f"Roll No: {roll_no} -> {student}")
    else:
        print("No Student Found With That Roll Number.")

elif choice == 3:
    if not records:
        print("No Student Records Found.")
    else:
        print("\nAll Student Records:")
        for roll_no, info in records.items():
            print(f"Roll No: {roll_no} -> {info}")

elif choice == 4:
    print("Exiting Student Database. Goodbye!")
    break

else:
    print("Invalid Choice! Please Select An Option Between 1 & 4.")

if __name__ == "__main__":
    student_database()

```

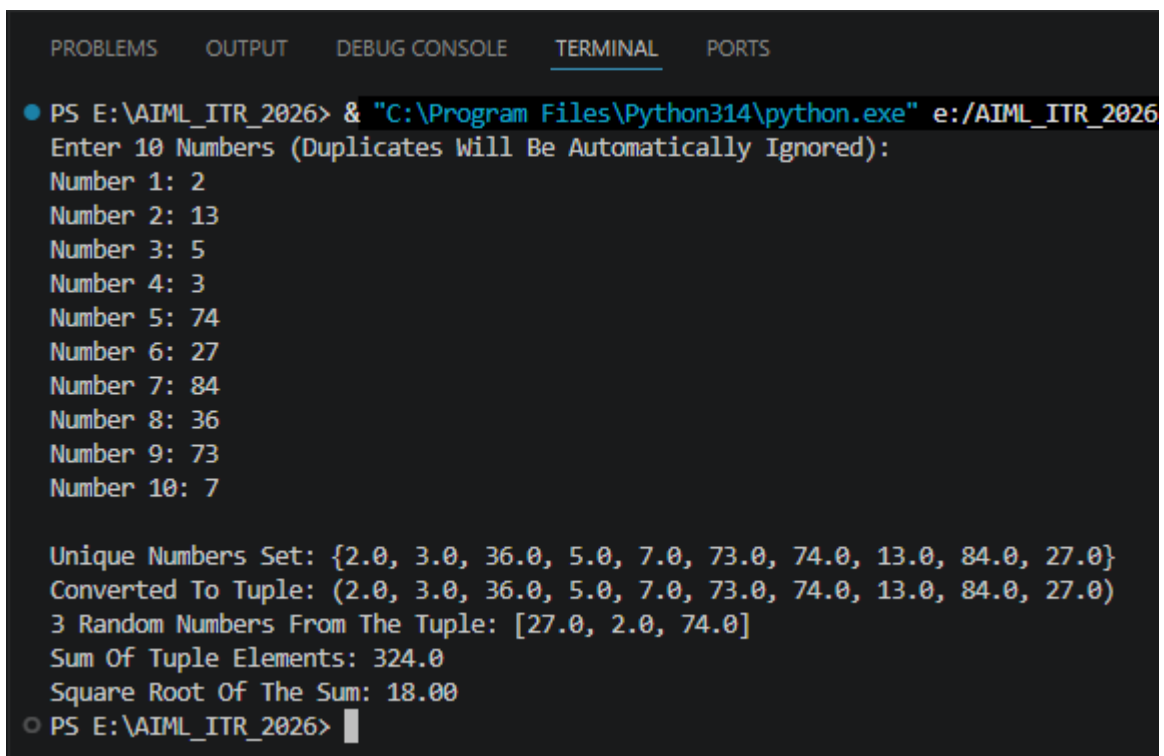
6. Sets + Tuples + Modules

Write a program that:

- Takes 10 numbers as input and stores unique numbers in a set.
- Converts that set into a tuple.
- Uses random module to pick and print 3 random numbers from the tuple.
- Uses math module to print the square root of the sum of the tuple elements.

Handle possible exceptions.

Output:



```
PS E:\AIML_ITR_2026> & "C:\Program Files\Python314\python.exe" e:/AIML_ITR_2026
Enter 10 Numbers (Duplicates Will Be Automatically Ignored):
Number 1: 2
Number 2: 13
Number 3: 5
Number 4: 3
Number 5: 74
Number 6: 27
Number 7: 84
Number 8: 36
Number 9: 73
Number 10: 7

Unique Numbers Set: {2.0, 3.0, 36.0, 5.0, 7.0, 73.0, 74.0, 13.0, 84.0, 27.0}
Converted To Tuple: (2.0, 3.0, 36.0, 5.0, 7.0, 73.0, 74.0, 13.0, 84.0, 27.0)
3 Random Numbers From The Tuple: [27.0, 2.0, 74.0]
Sum Of Tuple Elements: 324.0
Square Root Of The Sum: 18.00
PS E:\AIML_ITR_2026>
```

Program Code:

```
import random
import math

def main():
    number_set = set() # Unique Numbers Only

    print("Enter 10 Numbers (Duplicates Will Be Automatically Ignored):")
    count = 0
    while count < 10:
        try:
```

```

        value = float(input(f"Number {count + 1}: "))
        number_set.add(value)
        count += 1
    except ValueError:
        print("Invalid Input! Please Enter A Numeric Value.")

# Convert The Set Into A Tuple
number_tuple = tuple(number_set)
print(f"\nUnique Numbers Set: {number_set}")
print(f"Converted To Tuple: {number_tuple}")

# Use Random Module To Pick 3 Random Numbers From The Tuple
try:
    sample_size = min(3, len(number_tuple))
    random_numbers = random.sample(number_tuple, sample_size)
    print(f"3 Random Numbers From The Tuple: {random_numbers}")
except ValueError as e:
    print(f"Could Not Sample Random Numbers: {e}")

# Use Math Module To Print The Square Root Of The Sum Of Tuple Elements
try:
    total = sum(number_tuple)
    if total < 0:
        print("Cannot Compute Square Root Of A Negative Sum.")
    else:
        sqrt_value = math.sqrt(total)
        print(f"Sum Of Tuple Elements: {total}")
        print(f"Square Root Of The Sum: {sqrt_value:.2f}")
except Exception as e:
    print(f"An Error Occurred While Calculating The Square Root: {e}")

if __name__ == "__main__":
    main()

```

7. Lambda + range() + Lists + Exception Handling

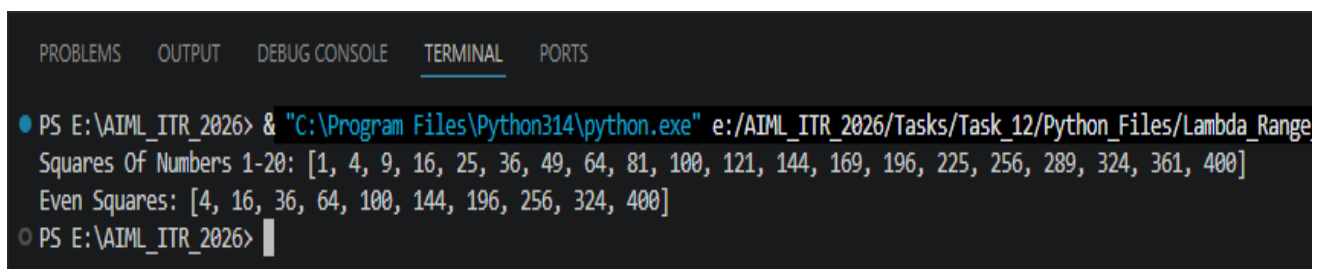
Create a lambda function to calculate the square of a number.

Write a program that:

- Uses range(1, 21) to generate numbers.
- Stores the squares of these numbers in a list using the lambda function.
- Prints only the even squares from the list.

Add exception handling if needed.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\AIML_ITR_2026> & "C:\Program Files\Python314\python.exe" e:/AIML_ITR_2026/Tasks/Task_12/Python_Files/Lambda_Range
Squares Of Numbers 1-20: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400]
Even Squares: [4, 16, 36, 64, 100, 144, 196, 256, 324, 400]
PS E:\AIML_ITR_2026>
```

Program Code:

```
def main():
    try:
        # Lambda Function To Compute The Square Of A Number
        square = lambda x: x ** 2
        squares=[]

        # Generate Numbers 1 To 20 & Store Their Squares
        for n in range(1,21):
            squares.append(square(n))
        print(f"Squares Of Numbers 1-20: {squares}")

        # Filter & Print Only The Even Squares
        even_squares=[]
        for s in squares:
            if s % 2 == 0:
                even_squares.append(s)
        print(f"Even Squares: {even_squares}")

    except Exception as e:
        # General Exception Handling In Case Something Unexpected Occurs
        print(f"An Error Occurred: {e}")

if __name__ == "__main__":
    main()
```

8. Tuples + Dictionaries + OOP

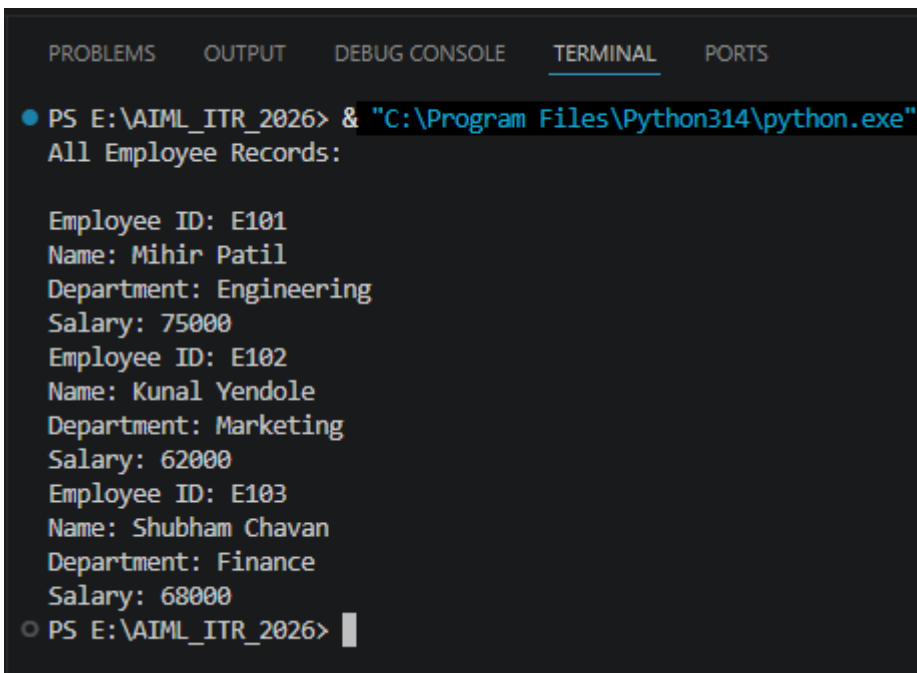
Create a class Employee with:

- Attributes: emp_id, name, details (a tuple containing department and salary).
- Method show_details() to print all information.

Create a dictionary where employee ID is the key and Employee object is the value.

Add 3 employees and display all using a loop.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS E:\AIML_ITR_2026> & "C:\Program Files\Python314\python.exe"
All Employee Records:

Employee ID: E101
Name: Mihir Patil
Department: Engineering
Salary: 75000
Employee ID: E102
Name: Kunal Yendole
Department: Marketing
Salary: 62000
Employee ID: E103
Name: Shubham Chavan
Department: Finance
Salary: 68000
○ PS E:\AIML_ITR_2026> |
```

Program Code:

```
class Employee:
    def __init__(self, emp_id, name, department, salary):
        self.emp_id = emp_id
        self.name = name
        # details Is A Tuple Containing Department & Salary
        self.details = (department, salary)

    def show_details(self):
        department, salary = self.details
        print(f"Employee ID: {self.emp_id}")
        print(f"Name: {self.name}")
        print(f"Department: {department}")
        print(f"Salary: {salary}")
```

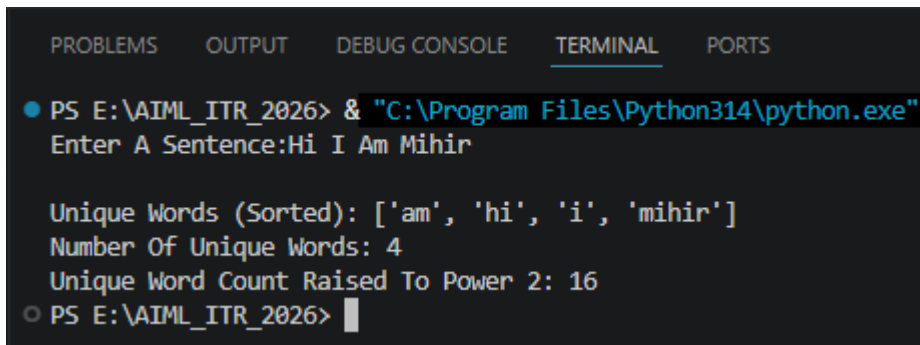
```
def main():  
    # Dictionary: employee ID (Key) -> Employee Object (Value)  
    employees = {}  
  
    # Add 3 employees  
    employees["E101"] = Employee("E101", "Mihir Patil", "Engineering", 75000)  
    employees["E102"] = Employee("E102", "Kunal Yendole", "Marketing", 62000)  
    employees["E103"] = Employee("E103", "Shubham Chavan", "Finance", 68000)  
  
    # Display All Employees Using A Loop  
    print("All Employee Records:\n")  
    for emp_id, employee in employees.items():  
        employee.show_details()  
  
if __name__ == "__main__":  
    main()
```

9. Strings + Sets + Exception Handling + Modules

Write a program that:

- Takes a sentence as input.
- Extracts all unique words (using set) from the sentence.
- Prints the unique words in sorted order.
- Uses math module to print the total number of unique words raised to power 2.
- Handle any possible exceptions gracefully.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS E:\AIML_ITR_2026> & "C:\Program Files\Python314\python.exe"
Enter A Sentence:Hi I Am Mihir

Unique Words (Sorted): ['am', 'hi', 'i', 'mihir']
Number Of Unique Words: 4
Unique Word Count Raised To Power 2: 16
○ PS E:\AIML_ITR_2026> 
```

Program Code:

```
import math

def main():
    try:
        sentence = input("Enter A Sentence:")

        if not sentence.strip():
            print("Error: The Sentence Is Empty.")
            return

        # Split The Sentence Into Words & Extract Unique Words Using A Set
        words = sentence.lower().split()
        unique_words = set(words)

        # Print Unique Words In Sorted Order
        sorted_words = sorted(unique_words)
        print(f"\nUnique Words (Sorted): {sorted_words}")

        # Use math Module To Raise The Count Of Unique Words To Power 2
        word_count = len(unique_words)
        result = math.pow(word_count, 2)
        print(f"Number Of Unique Words: {word_count}")
        print(f"Unique Word Count Raised To Power 2: {int(result)}")
```

```
except Exception as e:
    # Catch-All For Any Unexpected Errors
    print(f"An Error Occurred: {e}")

if __name__ == "__main__":
    main()
```


10. Mini Project - Comprehensive

Create a Smart Calculator & Data Manager program that combines multiple concepts:

- Use a while loop menu with these options:
- Basic Arithmetic (use functions + exception handling)
- Scientific Calculations (use math module)
- Generate Random Numbers (use random module)
- Store Results in Dictionary (with timestamp using string)
- View History (show stored results)
- Exit

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS E:\AIML_ITR_2026> & "C:\Program Files\Python314\python.exe"

***** Smart Calculator & Data Manager *****
1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Numbers
4. Store Last Result In Dictionary
5. View History
6. Exit
Enter Your Choice (1-6):1

*** Basic Arithmetic ***
Operations: + - * /
Choose An Operation (+, -, *, /):+
Enter First Number:1
Enter Second Number:2
Result: 1.0 + 2.0 = 3.0

***** Smart Calculator & Data Manager *****
1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Numbers
4. Store Last Result In Dictionary
5. View History
6. Exit
Enter Your Choice (1-6):2

*** Scientific Calculations ***
Options: 1) Square Root  2) Power  3) Factorial
Choose An Option (1-3):1
Enter A Number:36
Square Root Of 36.0 = 6.0
```

***** Smart Calculator & Data Manager *****

1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Numbers
4. Store Last Result In Dictionary
5. View History
6. Exit

Enter Your Choice (1-6):2

*** Scientific Calculations ***

Options: 1) Square Root 2) Power 3) Factorial

Choose An Option (1-3):2

Enter Base:4

Enter Exponent:2

$4.0 \wedge 2.0 = 16.0$

***** Smart Calculator & Data Manager *****

1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Numbers
4. Store Last Result In Dictionary
5. View History
6. Exit

Enter Your Choice (1-6):2

*** Scientific Calculations ***

Options: 1) Square Root 2) Power 3) Factorial

Choose An Option (1-3):3

Enter A Whole Number:5

Factorial Of 5 = 120

```
***** Smart Calculator & Data Manager *****
```

1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Numbers
4. Store Last Result In Dictionary
5. View History
6. Exit

```
Enter Your Choice (1-6):3
```

```
*** Generate Random Numbers ***
```

```
How Many Random Numbers Do You Want?3
```

```
Enter The Lower Bound:1
```

```
Enter The Upper Bound:7
```

```
Generated random numbers: [1, 7, 2]
```

```
***** Smart Calculator & Data Manager *****
```

1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Numbers
4. Store Last Result In Dictionary
5. View History
6. Exit

```
Enter Your Choice (1-6):4
```

```
Result Stored Under Timestamp: 2026-07-20 20:24:32
```

```
***** Smart Calculator & Data Manager *****
```

1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Numbers
4. Store Last Result In Dictionary
5. View History
6. Exit

```
Enter Your Choice (1-6):5
```

```
*** History ***
```

```
[2026-07-20 20:24:32] Random numbers [1, 7, 2]
```

```
***** Smart Calculator & Data Manager *****
```

1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Numbers
4. Store Last Result In Dictionary
5. View History
6. Exit

```
Enter Your Choice (1-6):6
```

```
Exiting Smart Calculator & Data Manager. Goodbye!
```

```
PS E:\AIML ITR 2026>
```

Program Code:

```
import math
import random
from datetime import datetime

# Dictionary To Store Results: {timestamp_string: result_description}
history = {}

def get_number(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Invalid Input! Please Enter A Numeric Value.")

def basic_arithmetic():
    print("\n*** Basic Arithmetic ***")
    print("Operations: + - * /")
    op = input("Choose An Operation (+, -, *, /):").strip()

    num1 = get_number("Enter First Number:")
    num2 = get_number("Enter Second Number:")

    try:
        if op == "+":
            result = num1 + num2
        elif op == "-":
            result = num1 - num2
        elif op == "*":
            result = num1 * num2
        elif op == "/":
            result = num1 / num2 # May Raise ZeroDivisionError
        else:
            print("Invalid Operation Selected.")
            return None

        print(f"Result: {num1} {op} {num2} = {result}")
        return f"{num1} {op} {num2} = {result}"

    except ZeroDivisionError:
        print("Error: Division By Zero Is Not Allowed.")
        return None

def scientific_calculations():
    print("\n*** Scientific Calculations ***")
    print("Options: 1) Square Root 2) Power 3) Factorial")
```

```

choice = input("Choose An Option (1-3):").strip()

try:
    if choice == "1":
        num = get_number("Enter A Number:")
        if num < 0:
            print("Cannot Calculate Square Root Of A Negative Number.")
            return None
        result = math.sqrt(num)
        print(f"Square Root Of {num} = {result}")
        return f"sqrt({num}) = {result}"

    elif choice == "2":
        base = get_number("Enter Base:")
        exponent = get_number("Enter Exponent:")
        result = math.pow(base, exponent)
        print(f"{base} ^ {exponent} = {result}")
        return f"{base}^{exponent} = {result}"

    elif choice == "3":
        num = int(get_number("Enter A Whole Number:"))
        if num < 0:
            print("Factorial Is Not Defined For Negative Numbers.")
            return None
        result = math.factorial(num)
        print(f"Factorial Of {num} = {result}")
        return f"{num}! = {result}"

    else:
        print("Invalid Option Selected.")
        return None

except Exception as e:
    print(f"An Error Occurred During Calculation: {e}")
    return None

def generate_random_numbers():
    print("\n*** Generate Random Numbers ***")
    try:
        count = int(get_number("How Many Random Numbers Do You Want?"))
        low = int(get_number("Enter The Lower Bound:"))
        high = int(get_number("Enter The Upper Bound:"))

        if count <= 0 or low > high:
            print("Invalid Range Or Count Provided.")
            return None

        random_numbers = []
        for _ in range(count):
            random_numbers.append(random.randint(low, high))

```

```

        print(f"Generated random numbers: {random_numbers}")
        return f"Random numbers {random_numbers}"

    except Exception as e:
        print(f"An Error Occurred:{e}")
        return None

def store_result(description):
    if description is None:
        print("Nothing To Store.")
        return

    # Create A Timestamp String As The Dictionary Key
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    history[timestamp] = description
    print(f"Result Stored Under Timestamp: {timestamp}")

def view_history():
    print("\n*** History ***")
    if not history:
        print("No Results Have Been Stored Yet.")
        return

    for timestamp, description in history.items():
        print(f"[{timestamp}] {description}")

def main():
    last_result = None # Keeps Track Of The Most Recent Result For Storing

    while True:
        print("\n***** Smart Calculator & Data Manager *****")
        print("1. Basic Arithmetic")
        print("2. Scientific Calculations")
        print("3. Generate Random Numbers")
        print("4. Store Last Result In Dictionary")
        print("5. View History")
        print("6. Exit")

        try:
            choice = int(input("Enter Your Choice (1-6):"))
        except ValueError:
            print("Invalid Input! Please Enter A Number Between 1 & 6.")
            continue

        if choice == 1:
            last_result = basic_arithmetic()
        elif choice == 2:

```

```
        last_result = scientific_calculations()
    elif choice == 3:
        last_result = generate_random_numbers()
    elif choice == 4:
        store_result(last_result)
    elif choice == 5:
        view_history()
    elif choice == 6:
        print("Exiting Smart Calculator & Data Manager. Goodbye!")
        break
    else:
        print("Invalid Choice! Please Select An Option Between 1 & 6.")

if __name__ == "__main__":
    main()
```